

BIO-INSPIRED ARTIFICIAL INTELLIGENCE

Constrained Evolutionary Algorithms

Dr. Giovanni Iacca

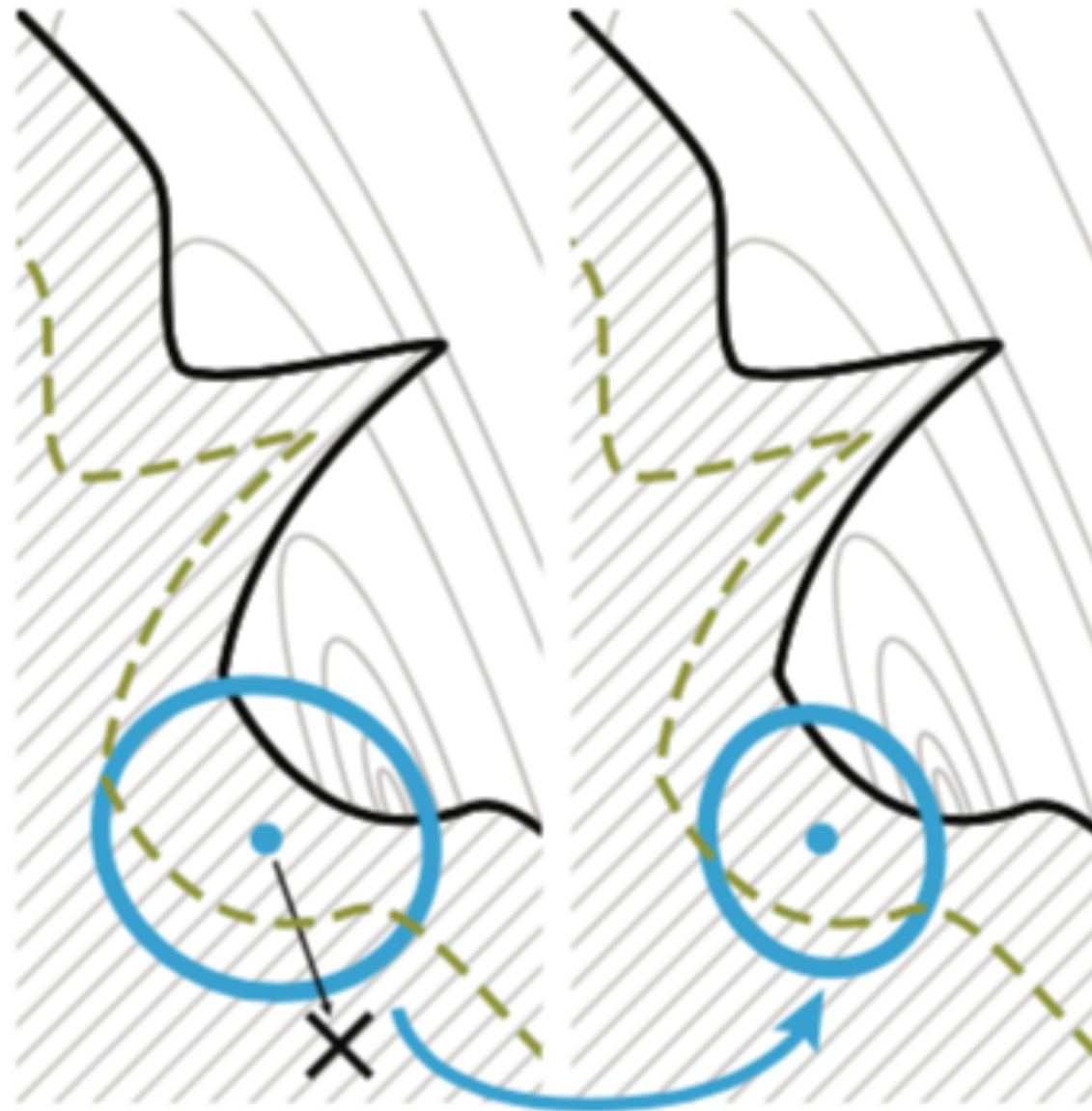
giovanni.iacca@unitn.it



UNIVERSITY OF TRENTO - Italy

**Information Engineering
and Computer Science Department**

Constrained Optimization



This lecture is based on the talk “Constraint-Handling Techniques used with Evolutionary Algorithms” given by Prof. C. Coello Coello at GECCO’18

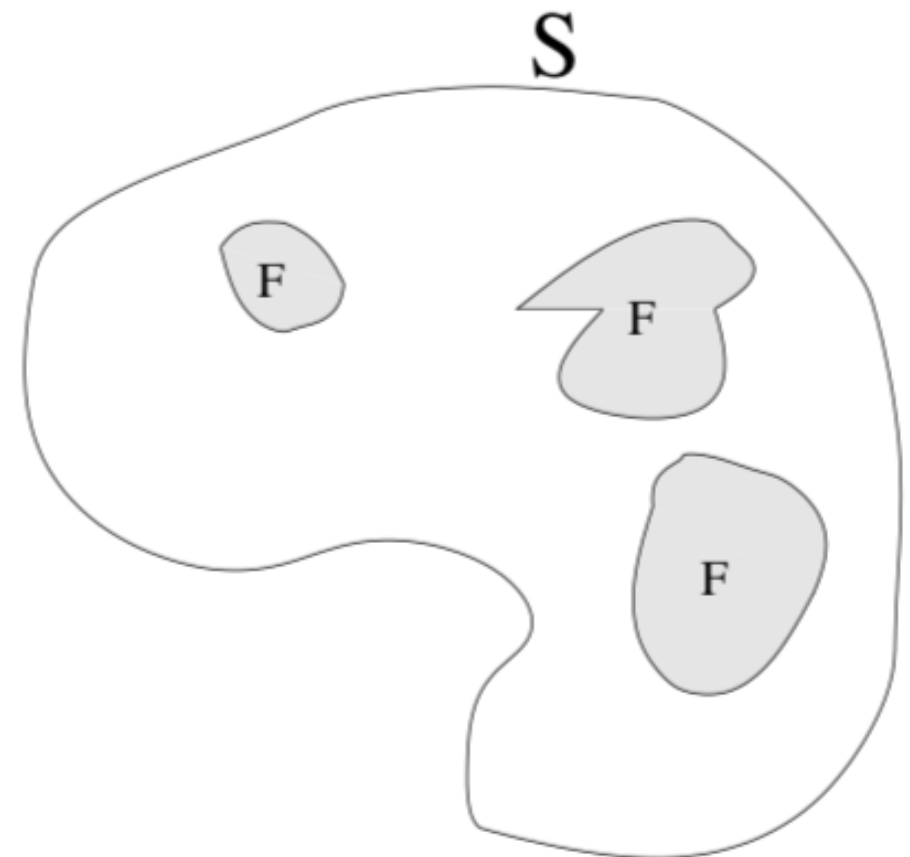
CONSTRAINED OPTIMIZATION

MOTIVATION

Recall the general optimization problem definition:

$$\begin{array}{ll} \text{find } x^* & \ni f(x^*) = \min_{x \in D} \{f(x)\} \\ \text{s.t:} & \begin{cases} g_j(x) \leq 0 & j = 1, 2, \dots, j_{\max} \\ h_k(x) = 0 & k = 1, 2, \dots, k_{\max} \end{cases} \end{array} \quad f(x) = \begin{cases} f_1(x) & : D \rightarrow F_1 \\ f_2(x) & : D \rightarrow F_2 \\ \dots & \\ f_m(x) & : D \rightarrow F_m \end{cases}$$

- Evolutionary Algorithms (EAs) are natively unconstrained (boundary constraint only) search techniques.
- However, most practical problems have a number of equality and inequality constraints.
- Several mechanisms have been proposed to allow EAs to deal with equality and inequality constraints, either linear or nonlinear:
 - **Penalty functions**
 - **Separation of constraints and objectives**
 - **Special representations and operators**
 - **Repair algorithms**
 - **Hybrid methods**



CONSTRAINED OPTIMIZATION

PENALTY FUNCTIONS

- **Death penalty:** when a certain solution violates a constraint, it is rejected
- **Static penalty:** the penalty function remains constant during the entire evolutionary process
- **Dynamic penalty:** the penalty function changes depending on the generation (normally the penalty factors are defined in such a way that they increase over time, i.e., generations)
- **Adaptive penalty:** penalty function which takes a feedback from the search process
- **Other penalty-based approaches**
 - ASCHEA
 - Stochastic Ranking

CONSTRAINED OPTIMIZATION

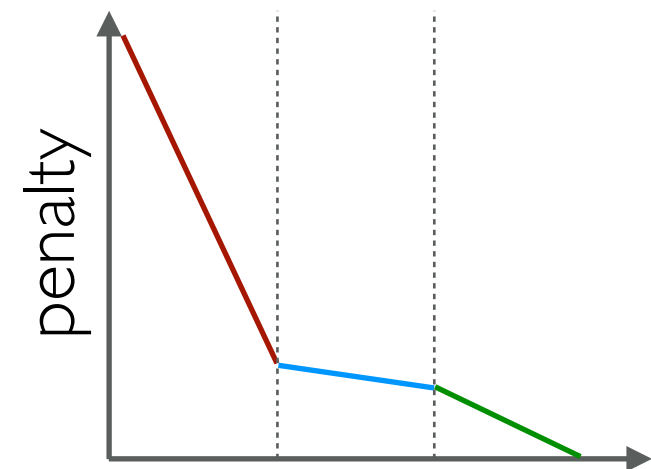
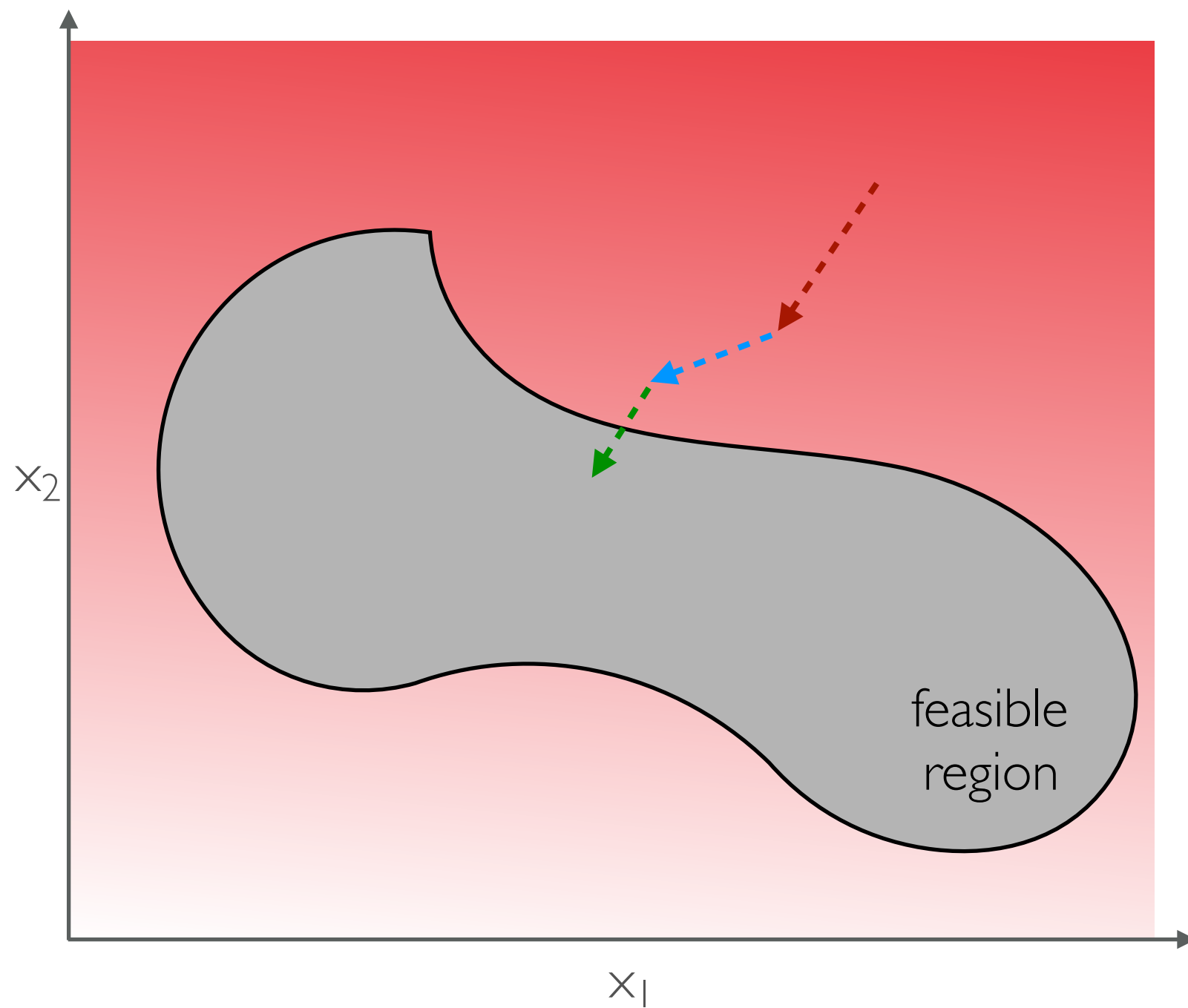
STATIC PENALTY

- Most common approach in the EA community to handle (particularly, inequality) constraints. Originally proposed (for classic optimization) by Richard Courant in the 1940s, later expanded.
- The idea of penalty functions is to transform a constrained optimization problem into an unconstrained one by adding (or subtracting) a certain value to/from the objective function, based on the constraint violation of the given solution.
- In classic optimization, two kinds of penalty functions are considered:
 - **Exterior methods:** we start with an infeasible solution and from there we move towards the feasible region (as the penalty decreases).
 - **Interior methods:** the penalty term is chosen such that its value is small at points away from the constraint boundaries, and tends to infinity as the constraint boundaries are approached. Then, if we start from a feasible point, the subsequent points generated will always lie within the feasible region since the constraint boundaries act as “barriers” during the optimization process.

In principle, both mechanisms can be used also in EAs!

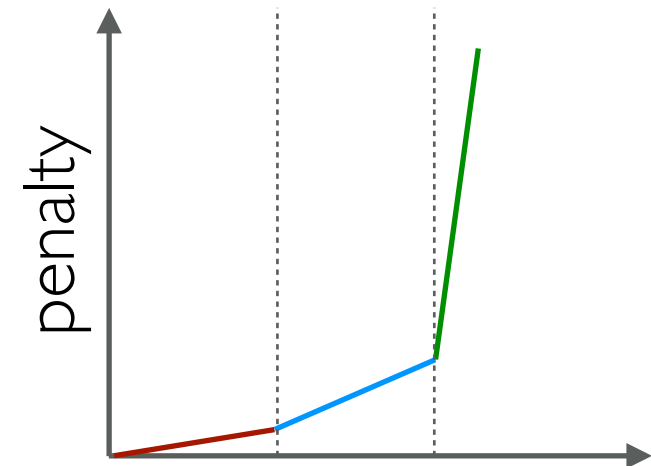
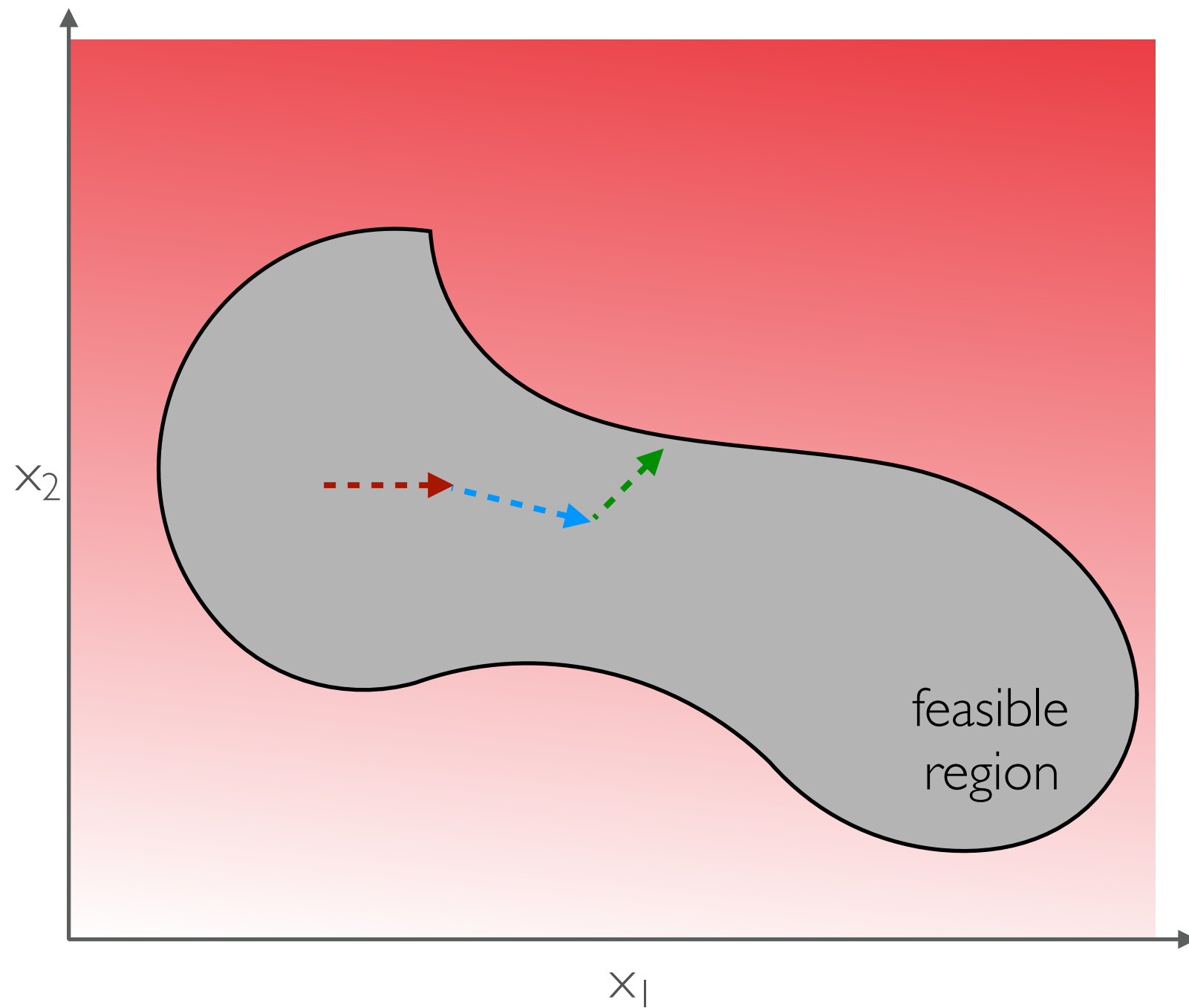
CONSTRAINED OPTIMIZATION

EXTERIOR METHODS



CONSTRAINED OPTIMIZATION

INTERIOR METHODS



CONSTRAINED OPTIMIZATION

STATIC PENALTY

- EAs normally adopt exterior penalty functions of the form:

$$\phi(\vec{x}) = f(\vec{x}) \pm \left[\sum_{i=1}^n r_i \times G_i + \sum_{j=1}^p c_j \times L_j \right]$$

where $\phi(\mathbf{x})$ is the new (expanded) objective function to be optimized, G_i and L_j are functions of the constraints $g_i(\mathbf{x})$ and $h_j(\mathbf{x})$, respectively, and r_i and c_j are positive constants normally called **penalty factors**.

- The most common form of G_i and L_j are:

$$G_i = \max[0, g_i(\vec{x})]^\beta$$

$$L_j = |h_j(\vec{x})|^\gamma$$

where β and γ are normally 1 or 2.

- Penalty functions can deal both with equality and inequality constraints, and the normal approach is to transform an equality constraint into an inequality one, of the form:

$$|h_j(\vec{x})| - \epsilon \leq 0$$

where ϵ is the tolerance allowed (an arbitrary very small value).

CONSTRAINED OPTIMIZATION

STATIC PENALTY

- Another way of applying static penalty is to add to the fitness of unfeasible solutions a penalty factor proportional to number of violated constraints (also called “level of infeasibility”) rather than proportional to the actual constraint violations $g_i(\mathbf{x})$ and $h_j(\mathbf{x})$:

$$\Phi(\mathbf{x}) = f(\mathbf{x}) + W \times \text{penalty}(\mathbf{x})$$

where W is a constant (arbitrarily big, positive or negative depending on minimization/maximization) and $\text{penalty}(\mathbf{x})$ is defined as follows:

$$\text{penalty}(x) = \sum_{j=1}^m \begin{cases} 1 & \text{for violated constraint} \\ 0 & \text{for satisfied constraint} \end{cases}$$

CONSTRAINED OPTIMIZATION

DYNAMIC PENALTY

- An example of this sort of approach was proposed by Joines and Houck [1994], where solutions are evaluated (at generation t) using:

$$\text{fitness}(\vec{x}) = f(\vec{x}) + (C \times t)^\alpha \times SVC(\beta, \vec{x})$$

$$SVC(\beta, \vec{x}) = \sum_{i=1}^n D_i^\beta(\vec{x}) + \sum_{j=1}^p D_j(\vec{x})$$

$$D_i(\vec{x}) = \begin{cases} 0 & g_i(\vec{x}) \leq 0 \\ |g_i(\vec{x})| & \text{otherwise} \end{cases} \quad 1 \leq i \leq n$$

$$D_j(\vec{x}) = \begin{cases} 0 & -\epsilon \leq h_j(\vec{x}) \leq \epsilon \\ |h_j(\vec{x})| & \text{otherwise} \end{cases} \quad 1 \leq j \leq p$$

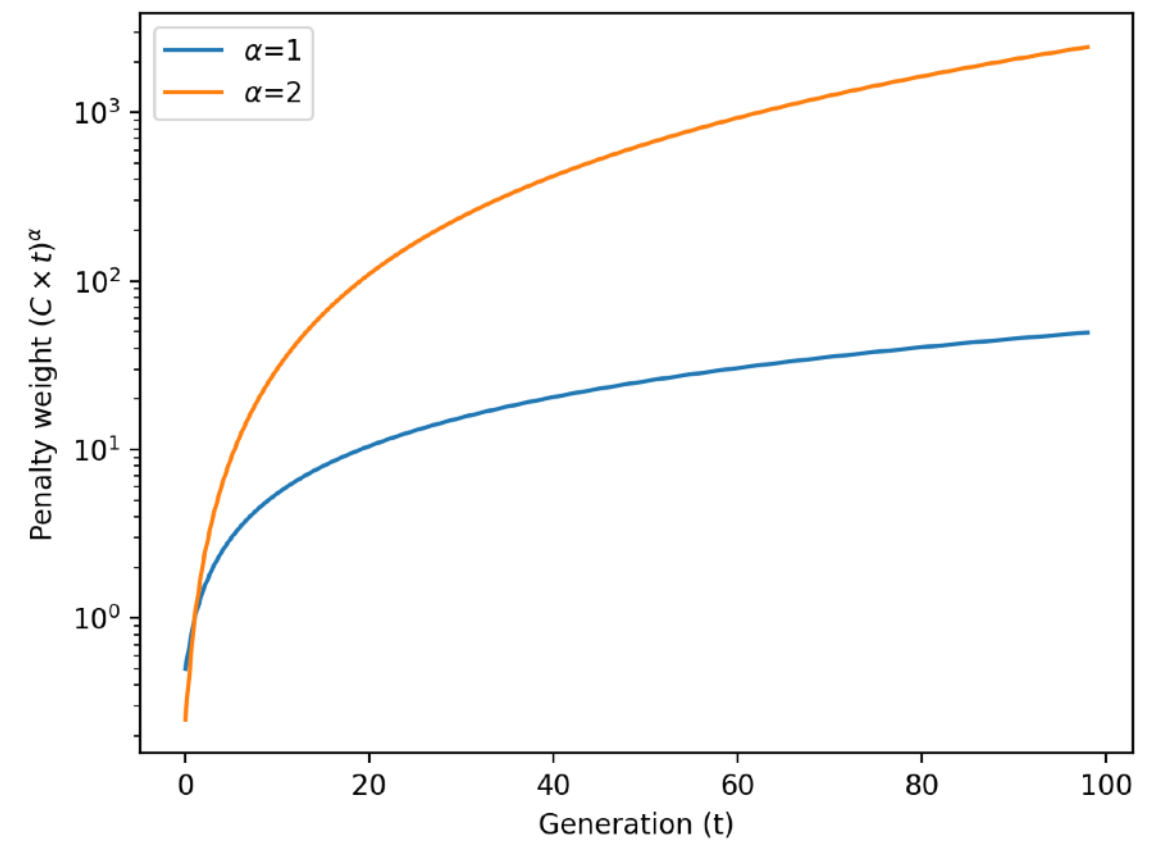
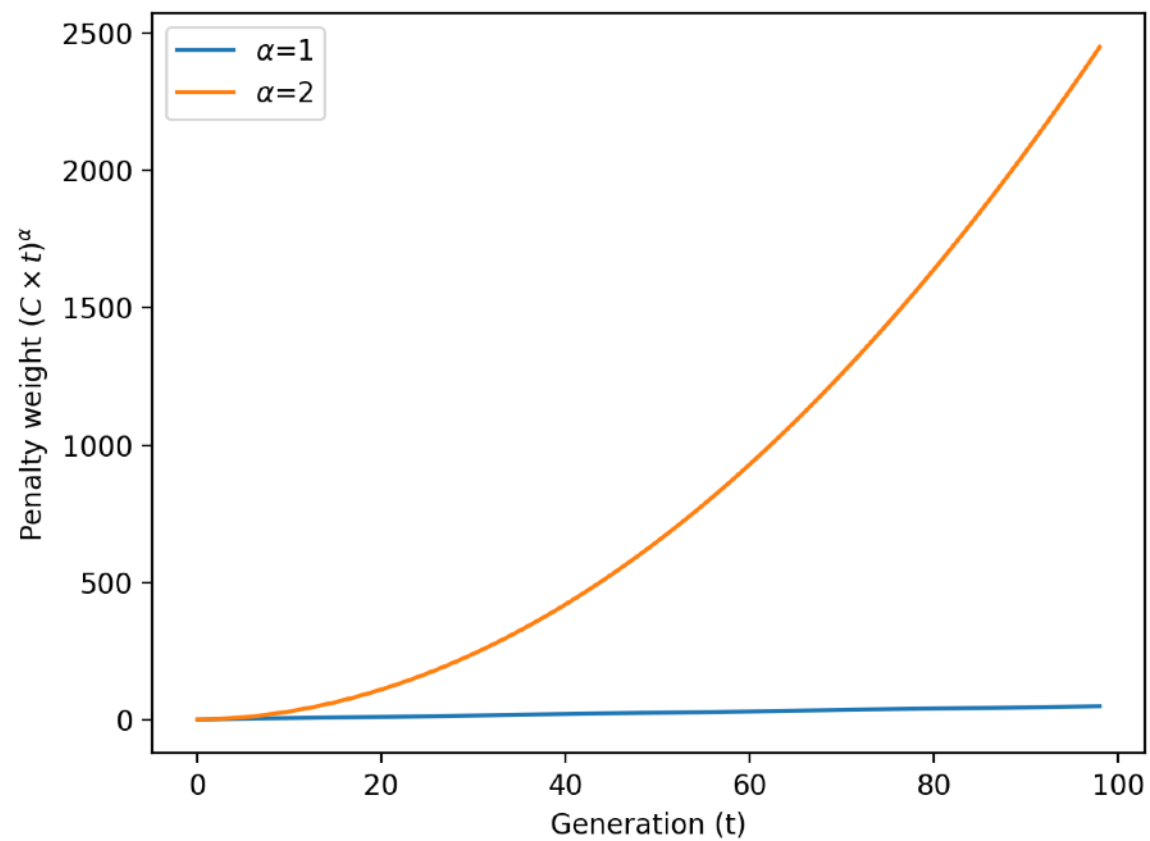
where C , α and β are user-defined constants (the authors used $C=0.5$, $\alpha=1$ or 2 , and $\beta=1$ or 2).

- This dynamic function increases the penalty as the evolutionary process proceeds (see next slide)

CONSTRAINED OPTIMIZATION

DYNAMIC PENALTY

Evolution of the penalty weight for $C=0.5$



CONSTRAINED OPTIMIZATION

ADAPTIVE PENALTY

- Bean and Hadj-Alouane [1992,1997] developed a method that uses a dynamic penalty function which takes a feedback from the search process. Each solution is evaluated by the formula:

$$\text{fitness}(\vec{x}) = f(\vec{x}) + \lambda(t) \left[\sum_{i=1}^n g_i^2(\vec{x}) + \sum_{j=1}^p |h_j(\vec{x})| \right]$$

where $\lambda(t)$ is updated at every generation t as follows:

$$\lambda(t+1) = \begin{cases} (1/\beta_1) \cdot \lambda(t), & \text{if case \#1} \\ \beta_2 \cdot \lambda(t), & \text{if case \#2} \\ \lambda(t), & \text{otherwise,} \end{cases}$$

where cases #1 and #2 denote situations where the best solution in the last k generations was always (case #1) or was never (case #2) feasible, $\beta_1, \beta_2 > 1$, $\beta_1 > \beta_2$, and $\beta_1 \neq \beta_2$ (to avoid cycling).

- In other words, the penalty component $\lambda(t+1)$ for the generation $t+1$ is decreased if all the best solutions in the last k generations were feasible or is increased if they were all infeasible. If there are some feasible and infeasible solutions tied as best in the population, then the penalty does not change.

CONSTRAINED OPTIMIZATION

MAIN ISSUES OF STATIC/DYNAMIC/ADAPTIVE PENALTY

- Even in the case of dynamic or adaptive penalty, the “ideal” penalty factors to be adopted in the penalty function are highly problem-dependent!
- If the penalty adopted is too high or too low, then the optimization process may not be efficient:
 - If the penalty is too high and the optimum lies at the boundary of the feasible region, the EA will be pushed inside the feasible region very quickly, and will not be able to move back towards the boundary between the feasible and infeasible region
 - If the penalty is too low, a lot of the search time will be spent exploring the infeasible region because the penalty will be negligible with respect to the objective function.
- Can we use better penalty-based approaches to solve these issues?
 - ASCHEA
 - Stochastic Ranking

CONSTRAINED OPTIMIZATION

OTHER PENALTY-BASED APPROACHES:ASCHEA

- The **Adaptive Segregational Constraint Handling Evolutionary Algorithm (ASCHEA)** was proposed by Hamida and Schoenauer [2000]. It uses Evolution Strategies and it is based on three main components:
 - An adaptive penalty function
 - A recombination operator guided by the constraints
 - A so-called “segregational” selection operator
- The **adaptive penalty function** is the following (in this case, assuming maximization):

$$fitness(\vec{x}) = \begin{cases} f(\vec{x}) & \text{if the solution is feasible} \\ f(\vec{x}) - penal(\vec{x}) & \text{otherwise} \end{cases}$$

$$penal(\vec{x}) = \alpha \sum_{j=1}^q g_j^+(\vec{x}) + \alpha \sum_{j=q+1}^m |h_j(\vec{x})|$$

where $g_j^+(\mathbf{x})$ is the positive part of $g_j(\mathbf{x})$ and α is the penalty factor adopted for all the constraints.

CONSTRAINED OPTIMIZATION

OTHER PENALTY-BASED APPROACHES: ASCHEA

- The penalty factor α is adapted based on the desired ratio of feasible solutions (with respect to the entire population) τ_{target} and the current ratio at generation t τ_t :

$$\begin{aligned} \text{if}(\tau_t > \tau_{target}) \quad & \alpha(t+1) = \alpha(t) / fact \\ \text{else} \quad & \alpha(t+1) = \alpha(t) * fact \end{aligned}$$

where $fact > 1$ and τ_{target} are user-defined parameters and

$$\alpha(0) = \left| \frac{\sum_{i=1}^n f_i(\vec{x})}{\sum_{i=1}^n V_i(\vec{x})} \right| * 1000$$

where $V_i(x)$ is the sum of the constraint violations of solution i

- The **recombination operator guided by the constraints** combines an infeasible solution with a feasible one when $\tau_t < \tau_{target}$. If $\tau_t > \tau_{target}$, then the recombination is performed in the traditional way (i.e., disregarding feasibility).
- The **segregational selection operator** aims to define a ratio τ_{select} of feasible solutions such that they become part of the next generation. The remaining solutions are selected in the traditional way based on their penalized fitness. τ_{select} is another user-defined parameter.

CONSTRAINED OPTIMIZATION

OTHER PENALTY-BASED APPROACHES: STOCHASTIC RANKING

- This approach was proposed by Runarsson and Yao [2000], and it consists of multi-membered Evolution Strategies that uses a penalty function and a selection based on a ranking process.
- The idea of the approach is to try to balance the influence of the objective function and the penalty function when assigning fitness to a given solution.
- An interesting aspect of the approach is that it doesn't require the definition of a penalty factor. Instead, the approach requires a user-defined parameter called P_f , which determines the balance between the objective function and the penalty function.
- The population is sorted using an algorithm similar to bubble-sort (which sorts the indexes of a list-based population) based on pairwise comparisons as follows:
 - if $u \sim U(0,1) < P_f$, the comparison of two solutions is done based only on the objective function
 - otherwise, the comparison is based on the sum of constraint violations
- Thus, P_f introduces the “stochastic” component to the ranking process, so that some solutions may get a good rank even if they are infeasible.

CONSTRAINED OPTIMIZATION

OTHER PENALTY-BASED APPROACHES: STOCHASTIC RANKING

- The value of P_f impacts the performance of the approach. The authors empirically found that $0.4 < P_f < 0.5$ produces the best results.
- This method requires relatively less fitness function evaluations than other methods.
- The approach is also easy to implement.

```
Begin
  For i=1 to N
    For j=1 to P-1
      u=random(0,1)
      If ( $\phi(I_j) = \phi(I_{j+1}) = 0$ ) or ( $u < P_f$ )
        If ( $f(I_j) > f(I_{j+1})$ )
          swap( $I_j, I_{j+1}$ )
        Else
          If ( $\phi(I_j) > \phi(I_{j+1})$ )
            swap( $I_j, I_{j+1}$ )
      End For
      If no swap is performed
        break
    End For
  End
```

CONSTRAINED OPTIMIZATION

SEPARATION OF CONSTRAINTS AND OBJECTIVES

Unlike penalty functions which combine the value of the objective function and the constraints of a problem to assign fitness, these approaches handle constraints and objectives separately.

Some examples:

- **Behavioral memory:** Schoenauer and Xanthakis [1993] proposed to satisfy, sequentially, the constraints of a problem.
- **Coevolution:** Use two populations (one handling objectives and one handling constraints) that interact with each other [Paredis, 1994].
- **Use of multi-objective optimization concepts:** The main idea is to redefine the single-objective optimization of $f(\mathbf{x})$ as a multi-objective optimization problem with $m+1$ objectives, where m is the total number of constraints. Then, any multi-objective evolutionary algorithm can be adopted, such as NSGA-2 [Deb et al., 2001, Coello et al., 2002] (*continues on next slide*)

CONSTRAINED OPTIMIZATION

SEPARATION OF CONSTRAINTS AND OBJECTIVES

- **Use of multi-objective optimization concepts:** the main technique of this kind is the so-called **constrained domination** (or **Pareto Ranking**), which extends the Pareto dominance definition to (also single-objective) constrained optimization, as follows:

For **single-objective optimization**, three cases for comparing two solutions are identified:

- Both are feasible → choose the solution with better objective function value
- One is feasible and the other is not → choose the feasible solution
- Both are infeasible → choose the solution with smaller overall constraint violation

For **multi-objective optimization**, the definition of domination is modified to include constraints, i.e. a solution i is said to constrained-dominate a solution j , if any of the following conditions is true:

- Solution i is feasible and solution j is not.
- Solution i and j are both infeasible, but solution i has a smaller overall constraint violation.
- Solution i and j are both feasible and solution i Pareto-dominates solution j .

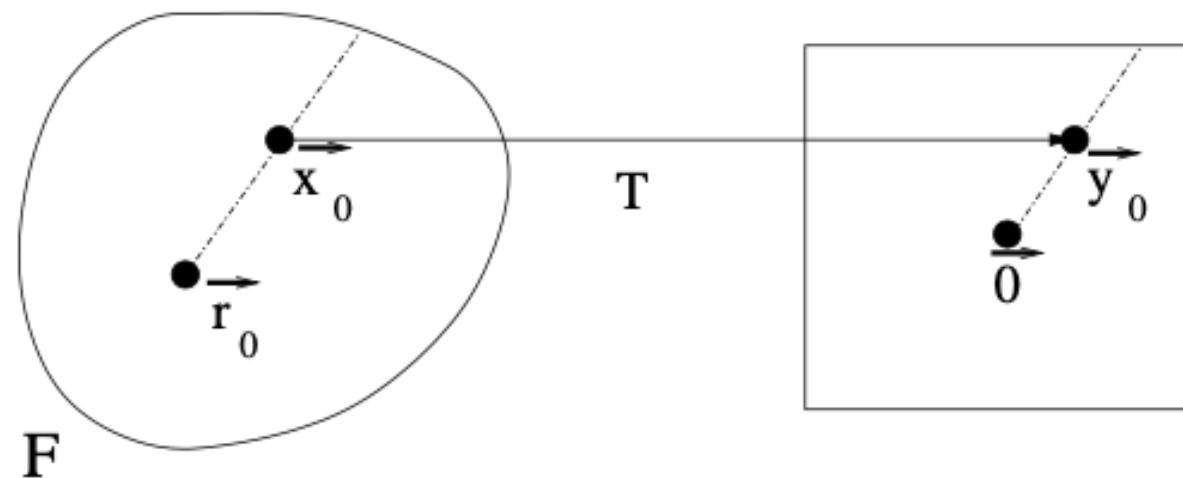
This means that:

- Any feasible solution has a better non-domination rank than any infeasible solution
- Feasible solutions are ranked according to their non-domination level (on objective functions only)
- Among infeasible solutions, the one with smaller constraint violation has better rank

CONSTRAINED OPTIMIZATION

SPECIAL REPRESENTATIONS AND OPERATORS

- On some particularly difficult problems, a generic representation scheme (e.g., the binary scheme used in the traditional GA) might not be appropriate. Special representation schemes and genetic operators have been designed for these cases. e.g. the **Random Keys** [Bean, 1992, 1994].
- Another intriguing idea is to transform the whole feasible region into a different shape that is easier to explore. The most important approach designed along these lines are the **Homomorphous Maps** [Koziel & Michalewicz, 1999]. This approach performs a homomorphous mapping between the feasible search space and an n-dimensional cube. The main idea of this approach is to transform the original problem into another that is easier to optimize by an Evolutionary Algorithm.



- In some cases, special (problem-dependent) genetic operators have also been proposed to produce offspring that lie on the boundary between the feasible and the infeasible region, or that are feasible by construction.

CONSTRAINED OPTIMIZATION

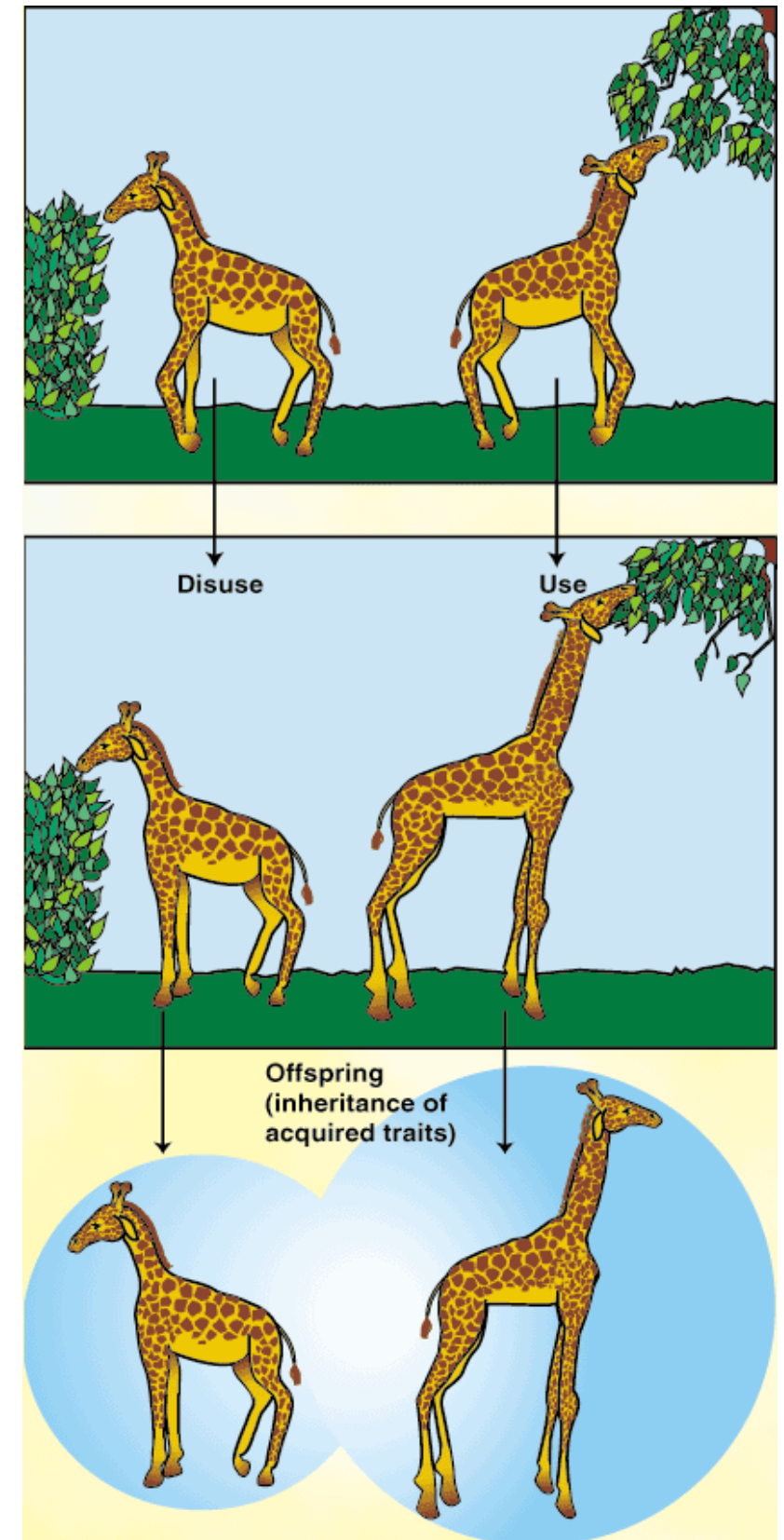
REPAIR ALGORITHMS

- This approach consists in devising a procedure that allows to transform an infeasible solution into a feasible one (i.e., “repair” the infeasible solution).
- Possible repair mechanisms are:
 - greedy algorithms (i.e., optimization algorithms that proceed through a series of steps by making at each step the best possible decision)
 - random algorithms (based on randomized perturbations of unfeasible solutions)
 - custom heuristics (problem-dependent, this is the case of most real-world problems)
- Repaired solutions can:
 - be used either for fitness evaluation only (“never replacing”)
 - replace the corresponding original unfeasible solutions in the population:
 - 1) always (“always replacing”)
 - 2) with some probability (e.g. “5% rule” for combinatorial problems)

NOTE: When applied, replacement can be seen as a form of **Lamarckian evolution**, which assumes that a solution improves during its lifetime and the resulting improvements are coded back into the chromosome.

CONSTRAINED OPTIMIZATION

Lamarckian evolution: acquired traits can be inherited from one generation to the other



CONSTRAINED OPTIMIZATION

REPAIR ALGORITHMS

Some examples of EAs based on repair mechanisms:

- **GENOCOP III** [Michalewicz & Nazhiyath, 1995]. This EA maintains two separate populations:
 - The first population consists of the so-called “search points”, i.e. trial solutions generated by means of genetic operators.
 - The second population consists of feasible “reference points”. Since these reference points are already feasible, they are used as reference to repair unfeasible search points.
- A **problem-dependent approach** was shown by Xiao et al. [1995], who used a repair algorithm to transform an infeasible path of a robot trying to move between two points in the presence of obstacles, so that the path would become feasible (i.e., collision-free). The repair algorithm was implemented through a set of carefully designed genetic operators that used knowledge about the domain to bring infeasible solutions into the feasible region in an efficient way.

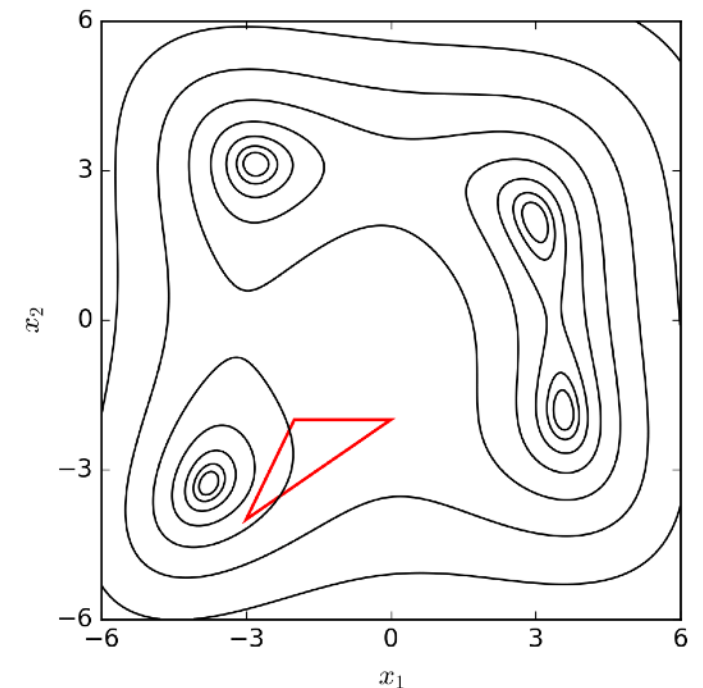
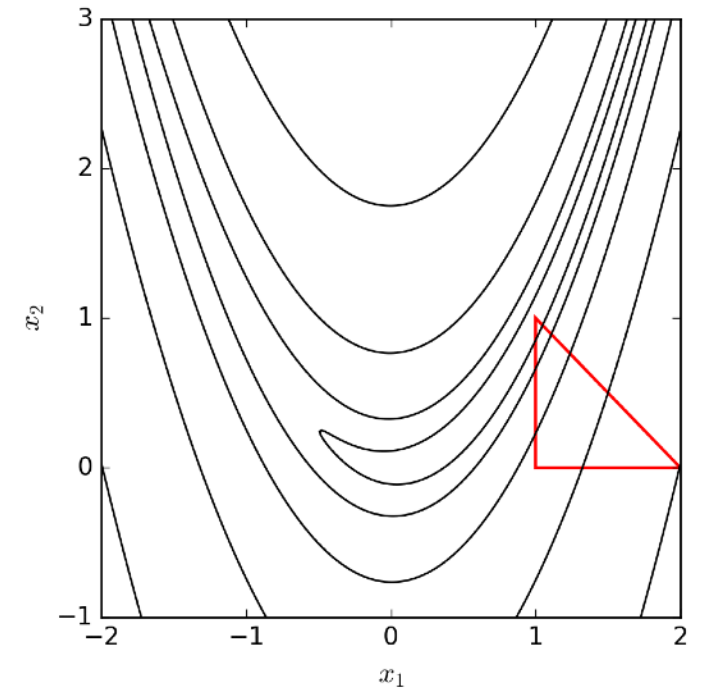
NOTE: When an infeasible solution can be easily (or at least at a low computational cost) transformed into a feasible solution, repair algorithms are a good choice. However this is not always possible and in some cases repair operators may introduce a strong bias in the search, harming the evolutionary process itself [Smith & Tate, 1993].

CONSTRAINED OPTIMIZATION

HYBRID METHODS

Another popular approach is to couple EAs with another search technique (e.g. another heuristic or a mathematical programming method). Some examples:

- **Genetic Algorithm + Artificial Immune System (AIS):** Hajela and Lee [1996] proposed the idea to adopt as *antigens* some feasible solutions and evolve by means of GA the *antibodies* (i.e., the infeasible solutions) so that they are similar (at a genotypic level) to the antigens.
- **ϵ constrained method:** The idea is to define a satisfaction level (ϵ) for the constraints of a problem, which are relaxed according to a lexicographic order. Takahama and Sakai [2006] used this approach in Differential Evolution with a gradient-based mutation operator, coupled to a modified version of Nelder and Mead's method. This algorithm represents currently one of the state-of-the-art EA for constraint optimization (but, it is very complex and very sensitive to the variation of some of its parameters).



https://en.wikipedia.org/wiki/Nelder-Mead_method

CONSTRAINED OPTIMIZATION

RECENT IDEAS

Ensemble of Constraint-Handling Techniques

The idea is to have several constraint-handling techniques, each with its own population and parameters (Mallipeddi and Suganthan [2010]). Each population produces its offspring and evaluates them. However, the offspring compete not only within each population, but also against offspring from the other populations. Thus, a certain offspring could be rejected by its population, while being accepted by another population. This intends to automate the selection of the best constraint-handling technique for a certain problem, based on the fact that none of them will be the best in all cases (remember the No-Free-Lunch theorem!).

Hybrid methods with Gradient-Based Information

Several new hybrid methods have been recently proposed, that incorporate gradient information, either from the fitness values or from the constraints. For example:

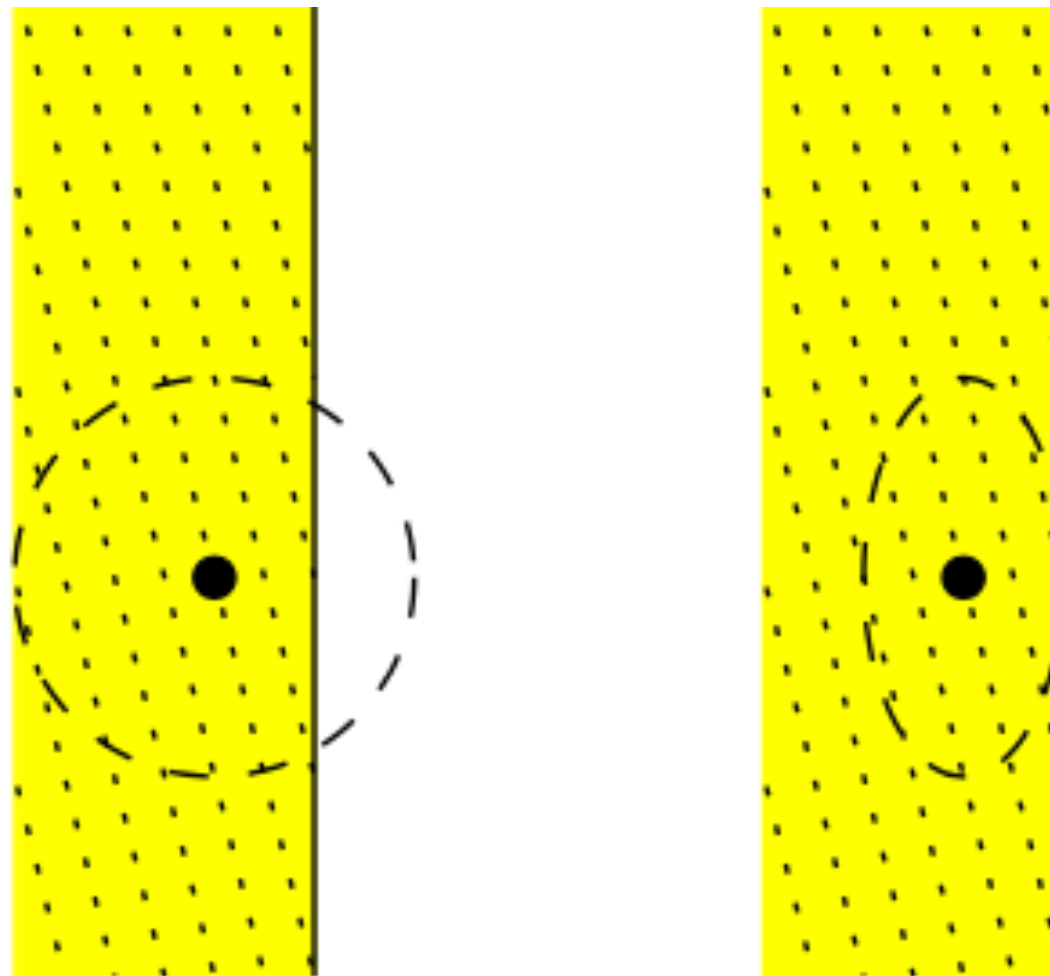
- Sun and Garibaldi [2010] proposed an Estimation of Distribution Algorithm combined with a gradient-based local search algorithm.
- Handoko et al. [2010] proposed a hybrid algorithm that combines a Genetic Algorithm, Sequential Quadratic Programming (SQP) with second-order functional approximations, and a technique for modeling the feasibility region through the use of Support Vector Machines (SVM).
- Hamza et al. [2014] proposed a consensus-based variant of a Genetic Algorithm which is combined with SQP, making use of gradient information from the constraints (rather than fitness).

CONSTRAINED OPTIMIZATION

RECENT IDEAS

Modified CMA-ES

In order to apply CMA-ES to constrained optimization problems, various solutions have been proposed, such as adaptive penalty functions (de Melo and Iacca [2014]) or modified Covariance Matrix update mechanisms to sample only in the feasible region (Arnold and Hansen [2012]).

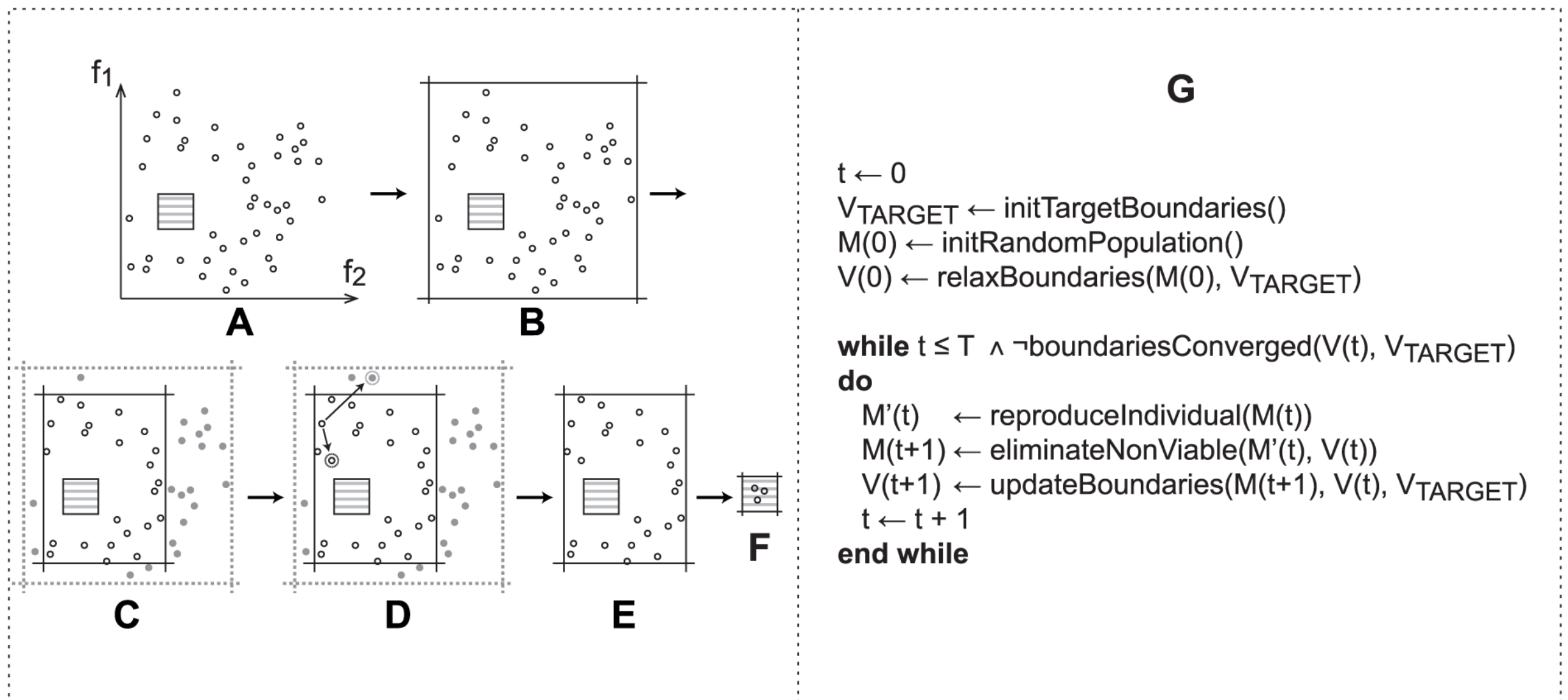


CONSTRAINED OPTIMIZATION

RECENT IDEAS

Viability Evolution

A different evolutionary paradigm based on elimination on “unviable” solutions w.r.t. progressively shrinking feasibility boundaries that are adapted at runtime [Maesani et al., 2014].



CONSTRAINED OPTIMIZATION

RECAP OF CONSTRAINT HANDLING TECHNIQUES (CHT)

	Technique	Pro	Con
Penalty-based	Death	Simple implementation	No gradual “push” towards feasibility
	Static	Simple implementation	Needs extra parameters/weights
	Dynamic	Adjusts penalty over time	Needs extra parameters/weights
	Adaptive	Uses feedback from search	Extra parameters, hard to design adapt. rules
	ASCHEA	Controls # feas. solutions	Extra parameters, needs large budget
	SR	Needs (relatively) small budget	1 extra parameter (P_f)
Separation of constraints and objectives	Various	No need for penalty, efficient handling also in MO cases	No control
Special representations & operators	Various	Can be very efficient on specific problems	Complexity, problem-specificity
Repair algorithms	Various	Efficient if repair is cheap	Complexity, problem-specificity, possible search bias
Hybrid methods	Various	Can be very efficient	Extra parameters, complexity
Recent ideas	Various	Can be very efficient	Extra parameters, complexity, may need gradient

CONSTRAINED OPTIMIZATION

CONCLUDING REMARKS

- For constrained optimization, various EAs have been used, with different constraint handling techniques. Based on empirical observations on a multitude of benchmark problems, Evolution Strategies and Differential Evolution seem to be more suited than Genetic Algorithms.
- The combination of parents and offspring in the selection process, i.e. the $(\mu+\lambda)$ strategy, tends to produce better performance (as for multi-objective optimization, elitism is needed to avoid losing the best feasible solution found so far).
- Diversity is important! Keeping only feasible solutions in the population is not always a good idea (indeed, some unfeasible solutions might be close to the boundary of the feasible region and may generate even better feasible solutions).
- Lot or research in this field is still going on!

Questions?